Esercitazione 8 - 16/11

Pochi esercizi, scelti per consolidare gli array prima di passare alle stringhe: ciascuno allena **una** idea precisa.

Argomenti:

- **invertire** un array, in tutto o solo in certe posizioni
- verificare una proprietà **sulle coppie di elementi vicini**
- **riempire** un array: con numeri casuali che rispettano una regola, e con numeri chiesti all'utente

Per ripassare array, `length`, scorrimento con `for` e la regola «le funzioni restituiscono un nuovo array» vedi i recap dell'esercitazione 7: Recap 1 · Gli array e Recap 2 · Array e funzioni.

Negli esercizi 1 e 2 scriviamo **programmi** (non funzioni) che lavorano sull'array `v`; negli esercizi 3 e 4 scriviamo **funzioni** che, come sempre, non modificano nulla e **restituiscono un nuovo array**. Nell'esercizio 1 anche il programma non modifica `v`: costruisce un array nuovo.

Indice degli esercizi

Esercizio	Argomento
Esercizio 1	Inversione di un array
Esercizio 2	Ogni 2 è seguito da un 3
Esercizio 3	Riempire un array con numeri casuali
Esercizio 4	Riempire un array con numeri chiesti all'utente
Esercizio 5	Copiare un array: perché non basta `b = a`

Esercizio 1

Dato un array `v`, scrivere un programma che costruisca un **nuovo array** `risultato` (senza modificare `v`) e lo stampi.

A. `risultato` contiene gli stessi elementi di `v` in ordine **inverso**. Ad esempio con `v = [1, 2, 3, 4]` si stampa `[4, 3, 2, 1]`.

B. In `risultato` gli elementi che si trovano in **posizione pari** (indici `0, 2, 4, ...`) sono presi in ordine inverso, mentre quelli in posizione dispari **restano dove sono**. Ad esempio:

- con `v = [1, 2, 3, 4, 5]` si stampa `[5, 2, 3, 4, 1]`
- con `v = [10, 11, 12, 13, 14, 15]` si stampa `[14, 11, 12, 13, 10, 15]`

Provate i programmi anche con un array di un solo elemento e con l'array vuoto.

Esercizio 2

Dato un array `v`, scrivere un programma che stampi `true` se **ogni** `2` presente nell'array è **immediatamente seguito** da un `3`, `false` altrimenti. Se non ci sono `2` la risposta è `true`; un `2` in ultima posizione non è seguito da niente, quindi dà `false`.

Provate il programma con questi array:

v	cosa stampa
`[1, 2, 3, 5, 2, 3]`	`true`
`[2, 4, 3]`	`false`
`[5, 2, 3, 2]`	`false`
`[1, 5, 7]`	`true`
`[2, 2, 3]`	`false`
`[]`	`true`

Esercizio 3

Funzioni che **costruiscono** un array di numeri casuali rispettando una regola. Ricordate che un intero casuale tra `min` e `max` (estremi inclusi) è `Math.floor(Math.random() * (max - min + 1)) + min`.

A. Scrivere la funzione `casualiPari(n, min, max)` che restituisce un array di `n` numeri casuali **pari** compresi tra `min` e `max`.

B. Scrivere la funzione `casualiDistinti(n, min, max)` che restituisce un array di `n` numeri casuali compresi tra `min` e `max` **tutti diversi tra loro**. Suggerimento: estraete un numero con un `do ... while` finché non ne trovate uno che non è già nell'array.

Provatele, ad esempio, con `casualiPari(8, 1, 20)` e `casualiDistinti(5, 1, 10)`.

Esercizio 4

Funzioni che **costruiscono** un array con numeri **chiesti all'utente**. Si legge con `prompt` e si converte con `Number`, come negli esercizi precedenti.

A. Scrivere la funzione `leggiArray(n)` che chiede all'utente `n` numeri e restituisce l'array che li contiene, nell'ordine in cui sono stati inseriti.

B. Scrivere la funzione `leggiFinoAZero()` che chiede numeri **finché l'utente non inserisce** `0` e restituisce l'array dei numeri inseriti, **senza** lo zero finale. Non si sa in anticipo quanti saranno.

C. Scrivere la funzione `leggiPositivi(n)` che chiede all'utente `n` numeri **positivi**: se l'utente ne inserisce uno non positivo, il programma lo richiede finché non è valido.

Per ciascuna scrivete poi un breve programma che la usa e stampa l'array letto.

Esercizio 5

Un array **non si copia** con l'assegnazione. Vediamo perché e come si fa.

A. Dite cosa stampa questo programma, **senza eseguirlo**. Poi spiegate con le vostre parole perché `a` è cambiato anche se abbiamo modificato solo `b`.

```
let a = [1, 2, 3];
let b = a;
b[0] = 99;
console.log(a);
console.log(b);
console.log(a == b);
```

B. Volevamo **conservare** `a` e lavorare su una copia. Scrivere la funzione `copia(v)` che restituisce un **nuovo array** con gli stessi elementi di `v`, e usarla per correggere il programma di A: dopo la modifica di `b`, `a` deve essere ancora `[1, 2, 3]`.

Ludovica Pannitto - Università di Pisa - CC BY-NC-SA 4.0